PLAYWRIGHT INTERVIEW PREP

Auto-waiting, waits & flaky tests

The topic almost every Playwright interview starts with. 8 real questions, how to structure your answer, and the exact points interviewers want to hear.

- 8 questions, Easy to Expert
- The 5-step answer structure
- A strong answer, word for word
- Mistakes & follow-ups to expect

WHY INTERVIEWERS ASK THIS

Does this person really know Playwright?

Playwright's biggest selling point is that it waits for you. So interviewers use this topic to find out whether you really understand Playwright, or are still writing Selenium habits in a new syntax: hard sleeps, checking `isVisible()` by hand, raising timeouts until a test goes green. A junior is expected to explain auto-waiting. A senior is expected to debug a flaky test in CI with traces and fix the root cause, not add retries.

PRACTISE THESE

8 questions, Easy to Expert

1. **What Is Auto-Waiting in Playwright?** — EASY
2. **Playwright: Auto-waiting and actionability** — EASY
3. **Playwright Auto-Wait vs Manual Assertions** — MEDIUM
4. **Playwright: Auto-Waiting vs page.waitForSelector()** — MEDIUM
5. **Playwright JS: Wait For a Specific Network Response** — MEDIUM
6. **How would you use Playwright Trace Viewer to investigate a test that fails intermittently in CI?** — HARD
7. **A Playwright test fails intermittently with a locator timeout, but increasing the timeout makes the failure disappear; how would you find the actual root cause?** — EXPERT
8. **How would you distinguish between a legitimate transient failure and a flaky test when Playwright retries a failed test?** — EXPERT

HOW TO ANSWER

Use this 5-step structure for every answer

1 **Define**
One sentence on what it is. "Before every action, Playwright waits until the element is actionable."

2 **Mechanism**
How it works: which checks run, what retries, which timeout applies, and what error you get when it gives up.

3 **When to use**
When the built-in waiting is enough, and when you still wait explicitly (a network response, a URL, a download).

4 **Pitfall**
The mistake people make with it. This is what separates you from someone who memorised a definition.

5 **Real example**
One line from your project: "After clicking Pay we assert the order number with toBeVisible(), with no fixed waits."

POINTS YOUR ANSWER MUST COVER (1/2)

What interviewers want to hear

1. **Locators are lazy.** A locator finds the element at the moment you act on it, so it never goes stale the way a stored Selenium element does. It is also **strict**: if it matches more than one element, the action throws.

2. **Actionability checks**: before `click()` Playwright waits until the element is attached, visible, stable (not animating), able to receive events and enabled. `fill()` also waits for it to be editable.

3. **Web-first assertions retry.** `await expect(locator).toBeVisible()` and `toHaveText()` keep checking until they pass or the expect timeout runs out (5 seconds by default).

4. `expect(await locator.isVisible()).toBe(true)` **does not retry.** It checks once and is a classic source of flakiness. Use the web-first assertion instead.

POINTS YOUR ANSWER MUST COVER (2/2)

What interviewers want to hear

5 `page.waitForTimeout()` **is a hard sleep.** It is meant for debugging, not for tests.

6 **Wait explicitly for things that are not elements**: `page.waitForResponse()` for an API call, `page.waitForURL()` or `expect(page).toHaveURL()` for navigation, the download event for files. Start waiting before the click that triggers it, or you can miss it.

7 **Know the timeouts**: each test has a timeout (30 seconds by default) and each expect has its own (5 seconds by default). Raising them hides problems; it doesn't fix them.

8 **Retries and traces**: with `retries` on, a test that fails and then passes is reported as flaky. Set `trace: 'on-first-retry'` and use the Trace Viewer to see exactly what the page looked like when it failed.

A STRONG ANSWER, WORD FOR WORD

Q: What is auto-waiting in Playwright?

> "Before Playwright performs an action like click or fill, it waits until the element is actionable: attached to the DOM, visible, stable, able to receive events and enabled. Locators are lazy, so the element is looked up again at action time and doesn't go stale. Assertions like expect(locator).toBeVisible() also retry until they pass or time out. So in our framework we don't use fixed waits at all. The only time we wait explicitly is for things that aren't elements: for example, when clicking Pay we start page.waitForResponse for the payment API before the click, then assert the order number with toBeVisible. The mistake I see most is expect(await locator.isVisible()).toBe(true), which checks only once and makes tests flaky."

About 45 seconds spoken. It defines auto-waiting, explains the mechanism, says when you still wait explicitly, names the pitfall, and gives a real example: the five-step structure above.

AVOID THESE

Mistakes that cost you marks

- ✗ Using `page.waitForTimeout(3000)` "to be safe"
- ✗ Checking `isVisible()` by hand instead of `await expect(locator).toBeVisible()`
- ✗ Raising timeouts or retries until a flaky test goes green
- ✗ Starting `waitForResponse` after the click, so the response is missed

BE READY FOR THESE

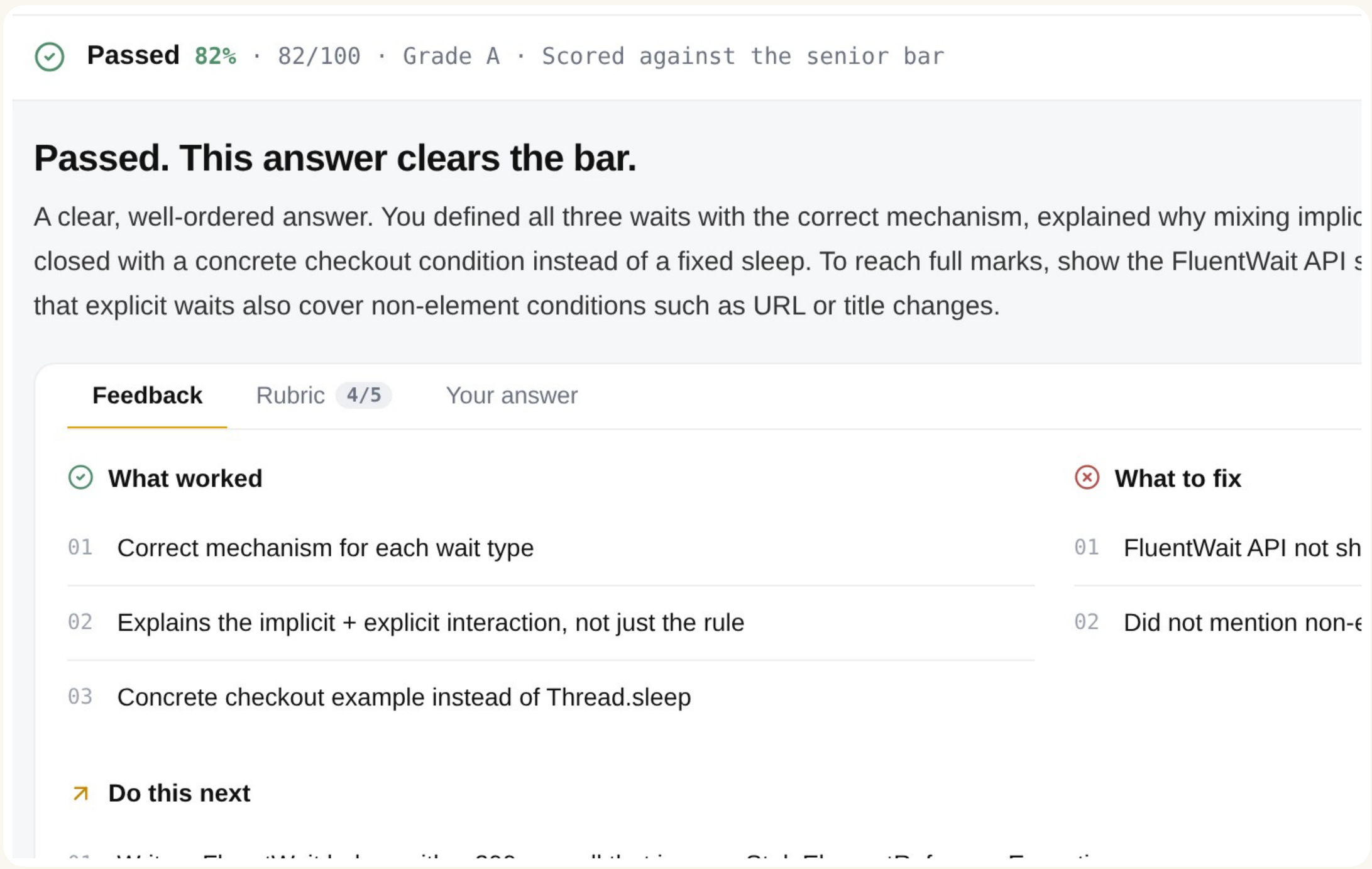
Follow-ups interviewers ask next

- ↻ Which checks does Playwright run before a click?
- ↻ What's the difference between `locator.isVisible()` and `expect(locator).toBeVisible()`?
- ↻ A click triggers an API call. How do you wait for that response without a race condition?
- ↻ A test passes on retry in CI. What do you do next?

PRACTISE OUT LOUD

Get an honest score on every answer

On ShapeMyInterview you answer these questions by speaking or writing code, just like the real round. Every answer is scored against the bar for your level, with **what worked** and **what to fix**.



YOUR TURN

Practise these 8 questions out loud

shapemyinterview.com



“One topic a week, practised out loud, beats reading ten topics the night before the interview.”

Naveen Khunteta
Founder, ShapeMyInterview

Follow for a new interview topic every week.

Next: Playwright locators, getByRole, test IDs, CSS or XPath